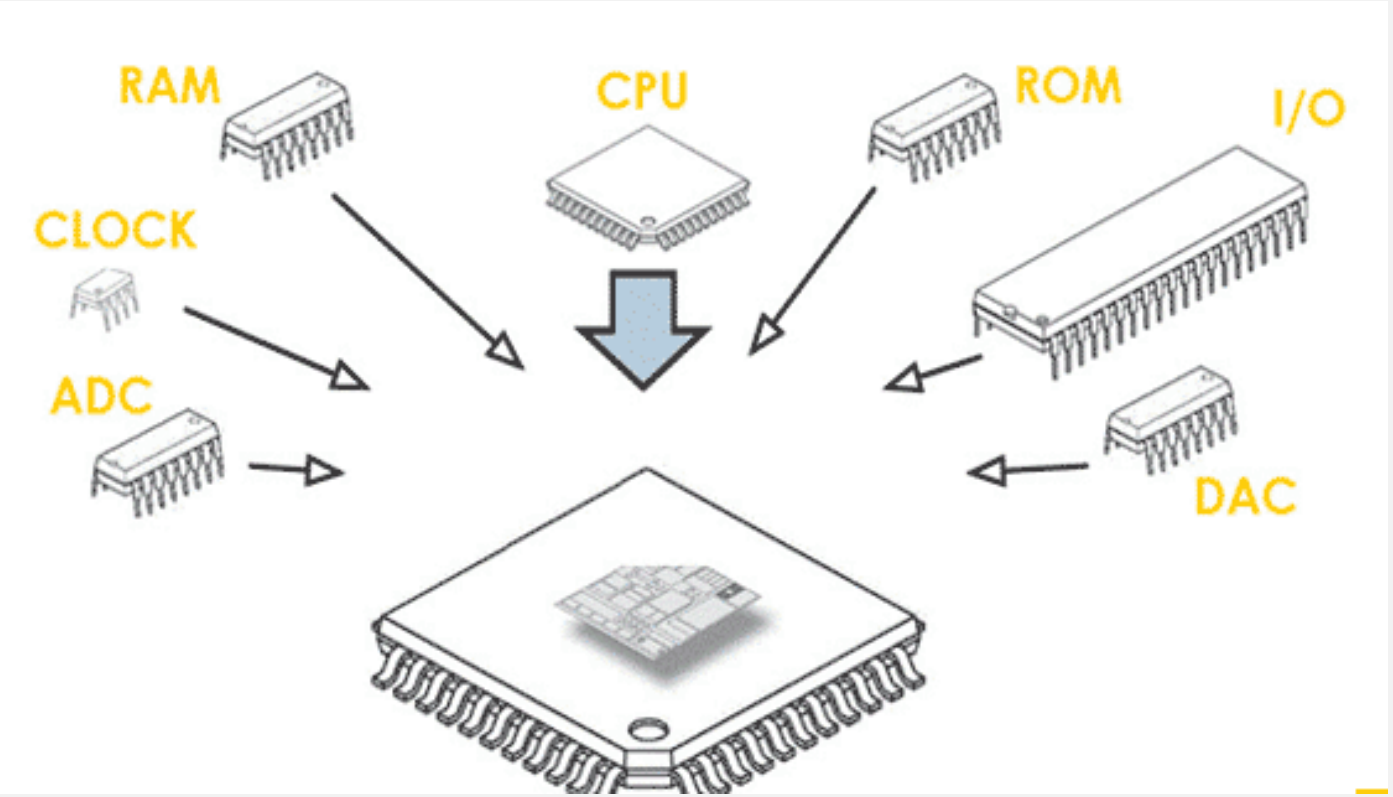
بسم الله الرحمن الرحيم

میکروکنترلر

میکروکنترلر Microcontrollerدر اصل یک چیپ الکترونیکی برنامه پذیر است که با اتصال قطعات مختلف در یک مدار الکترونیکی، اجزای یک کامپیوتر ساده را فراهم می کند. از میکروکنترلر برای ساخت، کنترل و مانیتورینگ انواع سیستم های الکترونیکی استفاده می شود که با برنامه ریزی واحد های میکروکنترلر و تجهیزات جانبی فعال می گردد به عنوان تعریفی دیگر در اصل یک کامپیوتر بسیار کوچک است که با جمع‌آوری دیگر قطعات الکترونیکی در یک فضای کوچک، اجزای یک کامپیوتر ساده را فراهم می کند.



اجزای مختلف میکروکنترلر

- CPU: در اصل همان مغز میکروکنترلر است، که وظیفه ی استخراج و پردازش داده ها، انجام محاسبات و وظایف اختصاص‌داده شده را بر عهده دارد

- حافظه: هر میکرو مقدار مشخصی از ROM، RAMو فلش ساخته می شود و تمام برنامه‌ها و داده‌ها میکروکنترلر در این واحد ذخیره می شوند. پردازنده CPUمعمولا از این واحد برای ذخیره‌سازی و دسترسی به اطلاعات استفاده میکند

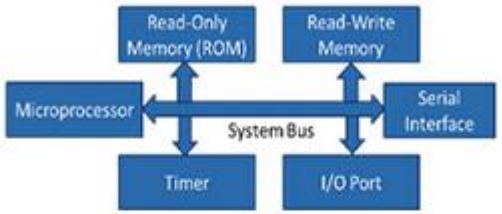
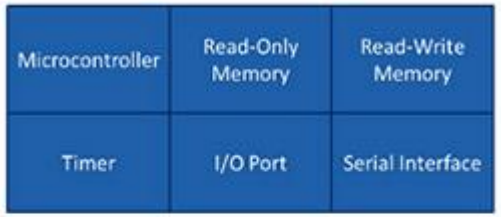
- ورودی و خروجی ها: پورت‌های ورودی و خروجی I/O در اصل راه ارتباطی هستند که تمامی میکروکنترلرها از آنها برای اتصال به وسایل جانبی، دریافت ورودی و نمایش خروجی استفاده می کنند

- تایمر: برای تولید پالس، اندازه‌گیری فرکانس، ساخت نوسانات و ... هر میکروکنترلر به یک یا چند تایمر (شمارنده) نیاز دارد. تا با کمک آنها، عملیات زمانبندی و شمارش را کنترل نماید

- مبدل آنالوگ به دیجیتال ADC و مبدل دیجیتال به آنالوگ DAC: بیشتر کمیت های اطراف ما که برای اندازه گیری، کنترل و شبیه سازی از آنها استفاده میکنیم، آنالوگ و پیوسته هستند اما میکروکنترلر تنها مقادیر دیجیتال را درک میکند. در نتیجه ما برای ایجاد یک زبان مشترک بین میکرو و کمیت هایی مانند گرما، نور و... از مبدل های ADCو DCAکمک می گیریم.

تفاوت بین میکروکنترلر و میکروپروسسور

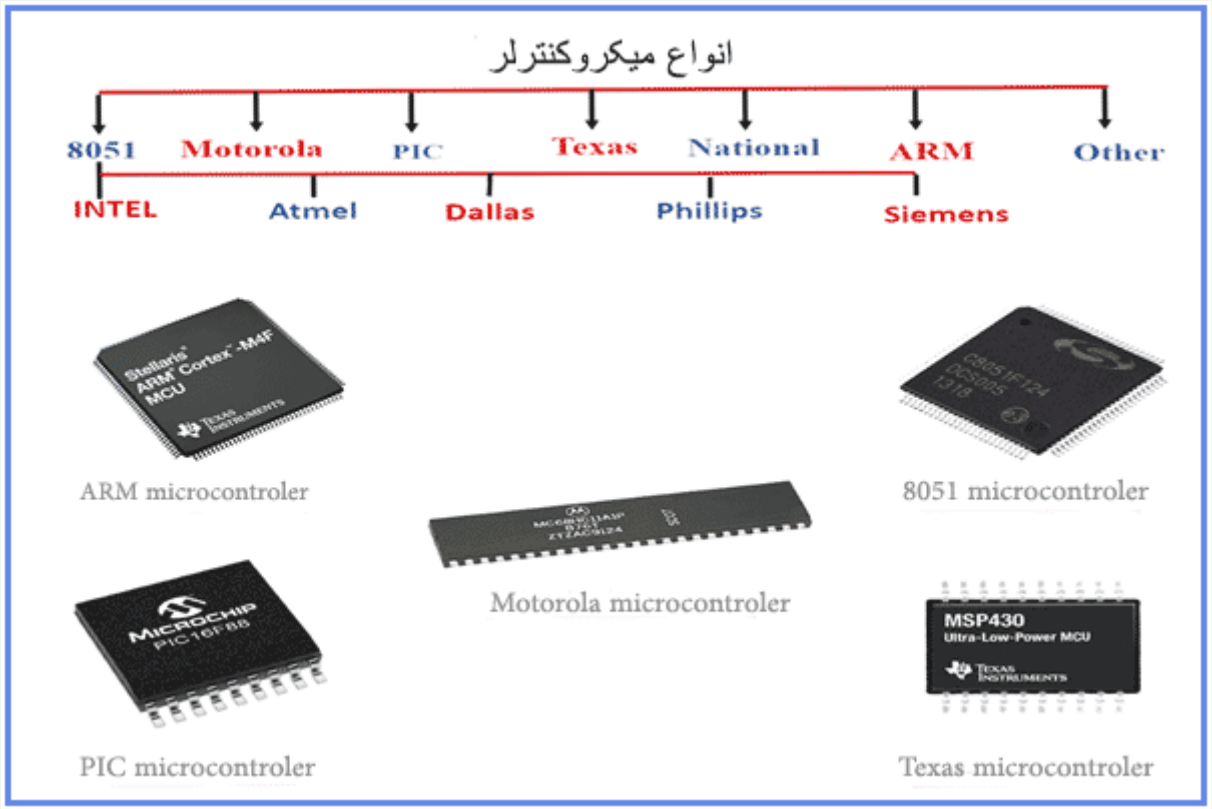
میکروپروسسور در اصل همان CPU یا مغز میکروکنترلر است که تمام عملیات محاسباتی و منطقی روی داده ها توسط آن انجام می شود.
اصلی ترین تفاوت بین میکروکنترلر و میکروپروسسور نیز در طریقه ی اتصال آنها به تجهیزات جانبی خارجی است.

Microprocessor میکروپروسسور	Micro Controller میکروکنترلر
	
میکروپروسسور قلب کامپیوتر است.	میکروکنترلر قلب سامانه‌های نهفته (Embedded system) است.
میکروپروسسور فقط یک پردازنده است؛ حافظه و واحدهای ورودی و خروجی باید به صورت جداگانه به آن اضافه شوند.	میکروکنترلر علاوه بر پردازنده، دارای حافظه و واحدهای ورودی و خروجی است.
به دلیل اینکه حافظه و واحدهای ورودی و خروجی باید به صورت جداگانه متصل شوند، مدار بزرگ‌تر خواهد بود.	به دلیل اینکه حافظه و واحدهای ورودی و خروجی درون آن قرار دارد، مدار کوچک‌تر خواهد بود.
نمی‌تواند در سیستم‌های فشرده استفاده شود.	به دلیل اینکه می‌تواند در سیستم‌های فشرده استفاده شود، بهینه‌تر است.
هزینه‌ی کلی سیستم زیاد است.	هزینه‌ی کلی سیستم کم است.
به دلیل اینکه تعداد قطعات خارجی زیاد است، مجموع توان مصرفی بیش‌تر است؛ بنابراین نمی‌توان برای تامین برق مصرفی آن، از منابع تغذیه‌ای مانند باتری استفاده کرد.	به دلیل اینکه تعداد قطعات خارجی کم است، مجموع توان مصرفی کم‌تر است؛ بنابراین می‌توان برای تامین برق مصرفی آن، از منابع تغذیه‌ای مانند باتری استفاده کرد.
به دلیل این که بیش‌تر قطعات خارج از آن قرار دارند، معمولا کندتر عمل می‌کند.	به دلیل داخلی بودن بیش‌تر قطعات، معمولا سریع‌تر عمل می‌کند.

انواع میکروکنترلر

در نتیجه بهتر است تا حد امکان از میکروکنترلر به جای میکروپروسسور، استفاده کنیم. زیرا ساخت هر دستگاهی با میکروکنترلر بسیار ارزان‌تر است.

اغلب میکروکنترلرها ویژگی های مشترک زیادی دارند زیرا همه ی آنها دارای یک حافظه درایو، پایه های ورودی و خروجی و توان مصرفی کم هستند. اما در جزئیاتی مانند تعداد پایه ها ، ابعاد، قیمت تمام شده و ... نیز با هم متفاوت اند.



انواع میکروکنترلر

انواع میکروکنترلرها براساس نوع کارکرد و مداری که در آن ها مورد استفاده قرار می‌گیرند به گروه زیر دسته بندی می شوند.

- میکروکنترلرهای AVR
- میکروکنترلرهای ARM
- میکروکنترلر های x-mega
- میکروکنترلرهای PIC
- میکروکنترلرهای ۸۰۵۱

همچنین میکروکنترلرها براساس حافظه، معماری، بیت ها، ساختار حافظه و مجموعه دستورالعمل ها به دسته بندی های مختلفی تقسیم می شوند.

البته پیشنهاد این است که در میان انواع میکروکنترلر، تا حد امکان از میکروکنترلرهای AVRاستفاده نمایید. مدل‌های میکرو کنترلر AVRتوسط شرکت Atmelروانه بازار شدند. این مدل از میکروکنترلرها از معماری RISKو CISKبرای تولید آن‌ها استفاده شده است. برخلاف میکروکنترلر ARM، قیمت مناسبی دارند و به راحتی در بازار یافت می شوند

انواع میکروکنترلر

در مورد ساختار پردازنده های CISCو RISCمی‌توان به این نکته اشاره کرد که دو نوع مختلف معماری مجموعه دستورالعمل هستند که برای طراحی ریزپردازنده‌های موجود در کامپیوترها استفاده می‌شوند. به طور کلی معماری CISC-کامپیوتر مجموعه دستورات پیچیده- با هدف کاهش تعداد دستورالعمل‌های مورد نیاز برای اجرای برنامه‌ها طراحی شده، در حالی که پردازنده RISC-رایانه مجموعه دستورات کاهش یافته- با تمرکز بر سادگی و سرعت اجرای دستورالعمل‌ها، بهینه‌سازی‌های قابل توجهی را در عملکرد به ارمغان آورد.

یک تفاوت CISCو RISCدر این است که RISCشامل دستورالعمل‌های ساده است که برای اجرای آنها نیاز به طی یک سیکل کامل داریم، در حالی که CISCشامل دستورالعمل‌های پیچیده است و ممکن است هر دستورالعمل به بیش از یک سیکل برای اجرا نیاز داشته باشند.

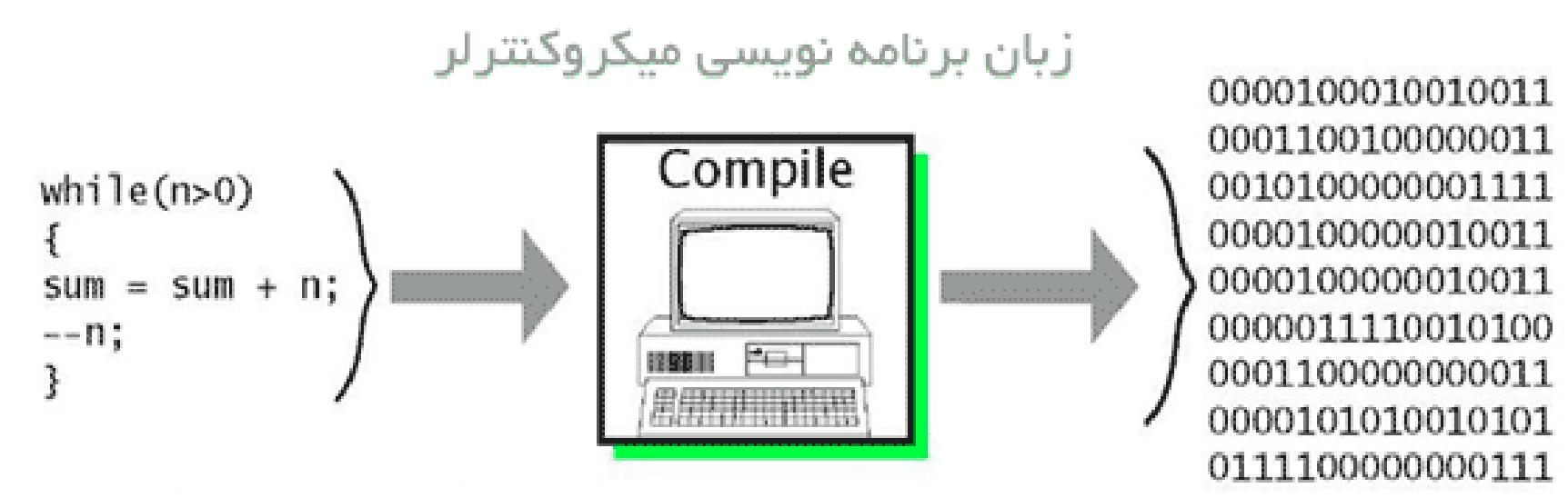
کاربرد میکروکنترلر

همانطور که اشاره شد، میکروکنترلر در اصل یک رایانه بسیار کوچک است که در هر پروژه و دستگاهی به طور همزمان نقش قلب و مغز مجموعه را ایفا میکند! بنابراین در حال حاضر در بیشتر لوازم خانگی و صنعتی مانند سیستم کنترل روشنایی، سیستم کنترل دما و آتش، سیستم های کنترل فرمان که در آنها اعمالی همچون اندازه گیری، ذخیره‌سازی، محاسبه، کنترل و نمایش اطلاعات انجام می شود، از میکروکنترلر استفاده شده است.



زبان برنامه نویسی میکروکنترلر

همانطور که میدانید میکروکنترلر تنها مقادیر دیجیتال را درک می کند، در نتیجه برای برقراری ارتباط با میکروکنترلر باید با زبان باینری (۰۱۰۱۰) با آنها صحبت کنیم.



بنابراین باید یک مترجم برای تبدیل دستورها به کد های باینری فراهم کنیم! این مترجم ها همان کامپایلر ها هستند که زبان های مختلف را به زبان ماشین تبدیل میکنند. در کشور ما کامپایلر های Bascom AVR، Code vision AVRو Atmel Studioبیشترین کاربرد را دارند.

برای برنامه نویسی میکروکنترلرها اغلب از زبان برنامه نویسی C، Assembly، Basicو ++ Cاستفاده می شود.

آردوینو

Arduinoچیست؟

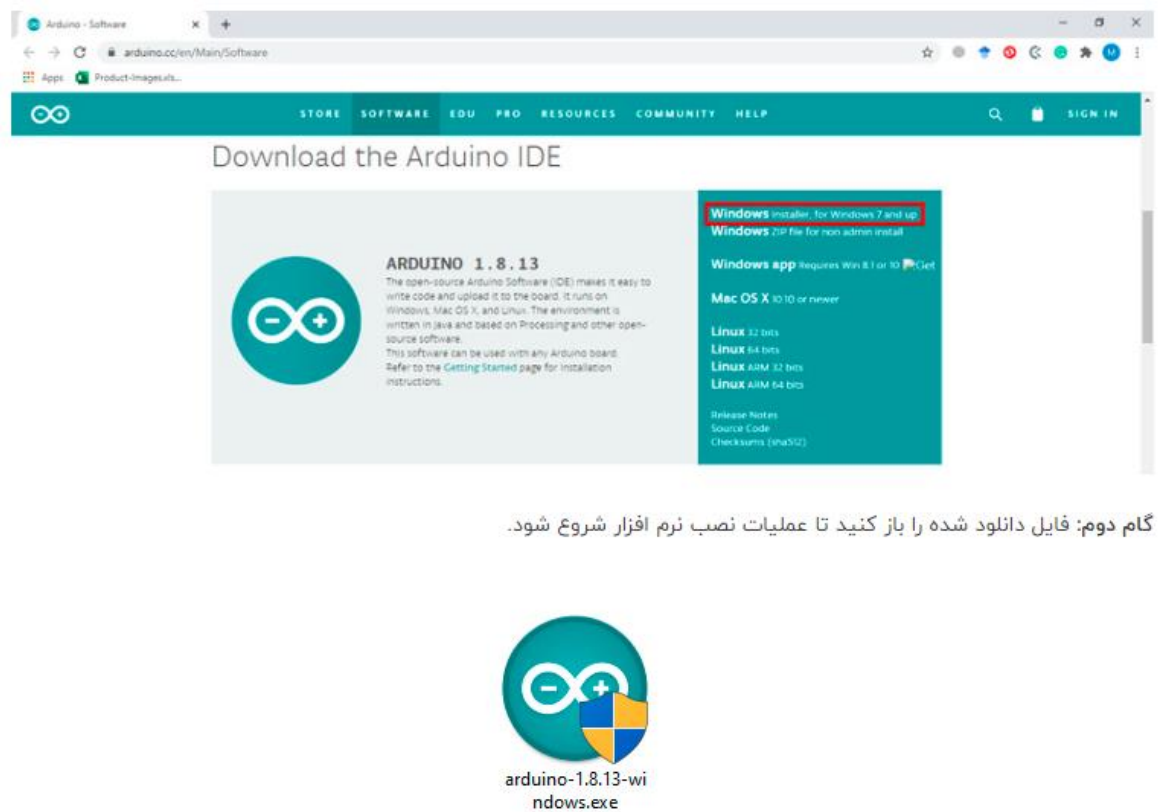
- Arduino یک پلتفرم متن‌بازOpen-source برای ساخت پروژه‌های الکترونیکی است.
- شامل سخت‌افزار و نرم‌افزار است که به راحتی می‌توان آن را برنامه‌نویسی کرد.
- برای افراد مبتدی و حرفه‌ای مناسب است.

نصب نرم افزار آردوینو

نرم افزار آردوینو (Integrated Development Environment) IDEیک نرم افزار ساده برای کدنویسی و پروگرام کردن بردهای خانواده ی آردوینو، ESP8266، ESP32و ... است. این نرم افزار توسط شرکت آردوینو طراحی و توسعه داده شده است و اصلی ترین نرم افزار برای برنامه نویسی و پروگرام کردن برد های خانواده ی آردوینو به شمار می رود.

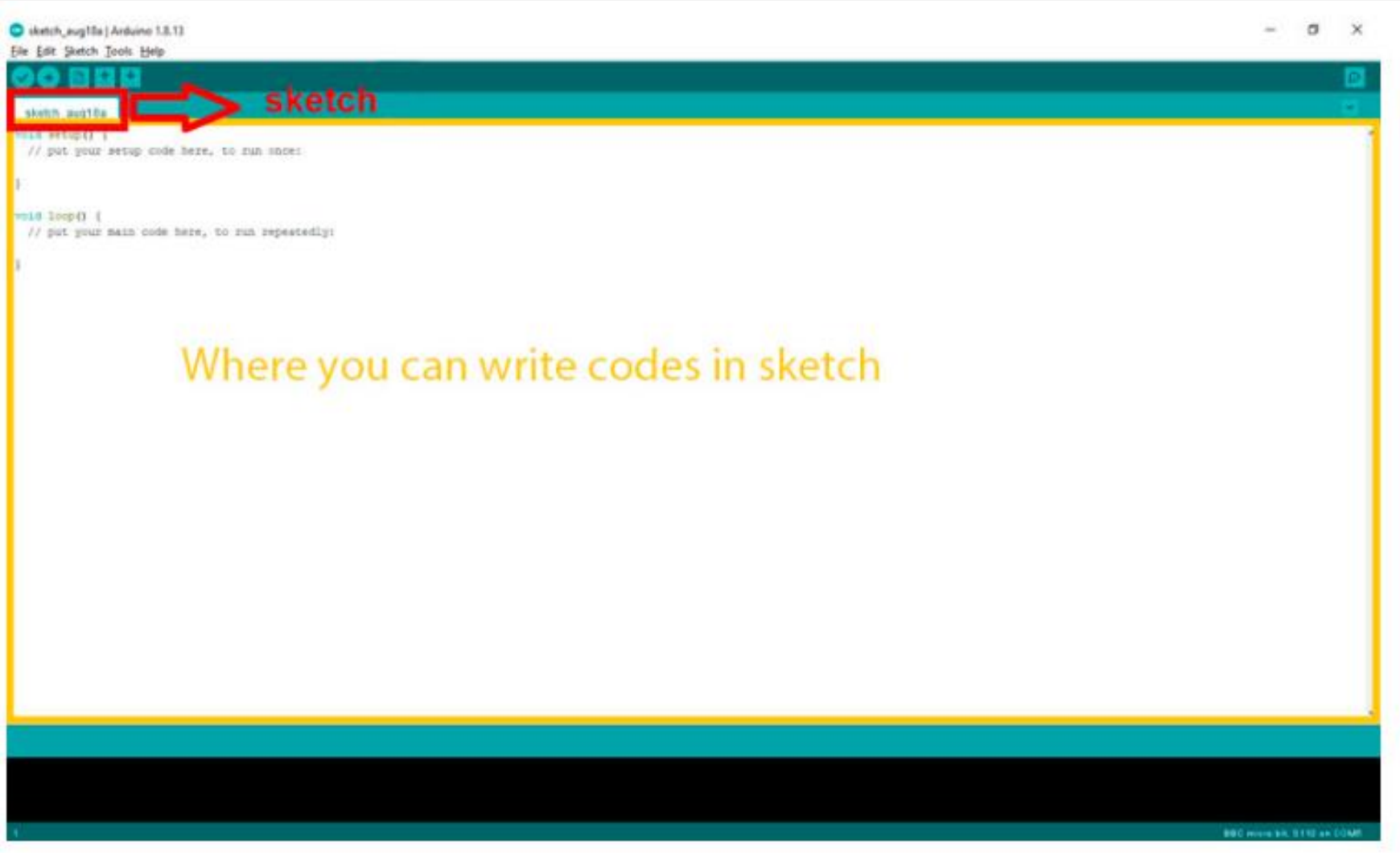
برای نصب این نرم افزار آخرین نسخه را از سایت https://www.arduino.cc/en/Main/Software دانلود کنید.

سپس آن را نصب کنید.



معرفی برخی از بخش های مختلف نرم افزار Arduino IDE

کدهای آردوینو در فایل هایی به نام اسکچ Sketchنوشته می شوند. این فایل ها با پسوند . inoدر حافظه ی کامپیوتر ذخیره می شوند. در شکل زیر محیط کار با یک اسکچ را مشاهده می کنید.



معرفی برخی از بخش های مختلف نرم افزار Arduino IDE

در بالای هر اسکچ، ۵ کلید وجود دارد.



Verify: صحت کد های نوشته شده را برسی می کند و سپس آن ها را کامپایل compileمی کند. دقت داشته باشید که این دکمه فقط کد شما را از نظر مطابقت با استانداردهای زبان برنامه نویسی می‌سنجد.



Upload: ابتدا کد شما را کامپایل کرده و سپس آن را به روی برد شما آپلود می کند. (با این کار، کد روی پردازنده ی برد نوشته می شود.)



New: یک اسکچ جدید ایجاد می کند.



Open: با این کلید می توانید اسکچ هایی که از قبل نوشته اید را باز کنید.



Save: اسکچ را ذخیره می کند.



معرفی برخی از بخش های مختلف نرم افزار Arduino IDE

تنظیمات و قابلیت های اضافه ی نرم افزار در منوهای File، Edit، Sketch، Toolsو Helpوجود دارد.

عملکرد زیربخش های این منو ها به شرح زیر است:

File

New: ساخت یک صفحه ی جدید (یک اسکچ جدید).

Open: باز کردن اسکچ هایی که از قبل ذخیره کرده ایم.

Open recent: لیستی از اسکچ هایی که اخیرا از آن ها استفاده کردید را دراختیارتان قرار می دهد و می توانید هرکدام از آن ها را که نیاز داشتید، باز کنید.

Sketchbook: اسکچ های موجود در اسکچ بوک را به شما نمایش می دهد. اسکچ بوک پوشه ای است که اسکچ های شما به صورت پیش فرض در آن ذخیره می شوند.

Examples: لیستی از مثال های آماده ی نرم افزار را در اختیارتان قرار می دهد.

Close: پنجره ی (اسکچ) فعلی را می بندد.

Save: اسکچ را با نام فعلی آن ذخیره می کند. اگر اسکچ شما از قبل نامگذاری نشده باشد، به صورت خودکار پنجره ی Save asبرای شما باز می شود.

Save as: به شما اجازه می دهد تا اسکچ فعلی را با نامی دیگر و در محلی جدید ذخیره کنید.

Page setup: تنظیمات صفحه را قبل از پرینت شدن نشان می دهد.

Print: پرینت گرفتن از پنجره ی نرم افزار.

Preferences: در این بخش، به قسمتی از تنظیمات نرم افزار دسترسی دارید. از این بخش می توانید زبان، سایز فونت و ... را تغییر دهید.

Quit: تمام پنجره های باز نرم افزار را می بندد.

معرفی برخی از بخش های مختلف نرم افزار Arduino IDE

Edit

Undo/Redo: به یک پله قبل / بعد می روید و عملیاتی که در مرحله ی قبل / بعد انجام داده اید، به شما نمایش داده می شود.
Cut: متن انتخاب شده را از آن محل پاک می کند و آن را در حافظه ی Clipboardذخیره می کند.
Copy: از متن انتخاب شده یک نسخه ی کپی ایجاد کرده و آن را در حافظه ی Clipboardذخیره می کند.
Copy for Forum: کدهای نوشته شده در اسکچ را به صورتی که برای استفاده در فروم (انجمن) مناسب است، کپی کرده و در حافظه ی Clipboardذخیره می کند.
Copy as HTML: کدهای نوشته شده در اسکچ را با فرمت HTMLکپی کرده و در حافظه ی Clipboardذخیره می کند.
Paste: محتوای موجود در حافظه ی Clipboardرا در محل مورد نظر قرار می دهد.
Comment/Uncomment: بخش انتخاب شده از کد شما را به صورت کامنت (یادداشت) در می آورد. با زدن مجدد این کلید، بخش مورد نظر از حالت کامنت خارج می شود.
Increase/Decrease: تورفتگی ابتدای خطوط انتخاب شده را افزایش / کاهش می دهد.
Find: متن مورد نظر شما را در اسکچ فعلی جستجو می کند. همچنین تنظیمات دیگری مثل جایگزین کردن یک متن با متنی دیگر را در اختیار شما قرار می دهد.
Find next: اگر در بخش Findعبارتی مشخص شده باشد، در بعد از مکان فعلی cursorبه دنبال آن می گردد.
Find previous: اگر در بخش Findعبارتی مشخص شده باشد، در قبل از مکان فعلی cursorبه دنبال آن می گردد.

معرفی برخی از بخش های مختلف نرم افزار Arduino IDE

Sketch

Verify/Compile: کدهای نوشته شده در اسکچ را از نظر ساختار برنامه نویسی بررسی کرده و اگر بدون خطا بود، آن را کامپایل می کند.
Upload: ابتدا کد را کامپایل کرده و سپس فایل باینری ایجاد شده را روی پردازنده آپلود می کند.
Upload using programmer: بوت لودر را بازنویسی می کند و شما قادر خواهید بود تا از تمام حافظه ی flashپردازنده استفاده کنید. دقت داشته باشید که با این کار فقط به حافظه ی flashدسترسی دارید و قادر به برنامه ریزی فیوزبیت ها نیستید.

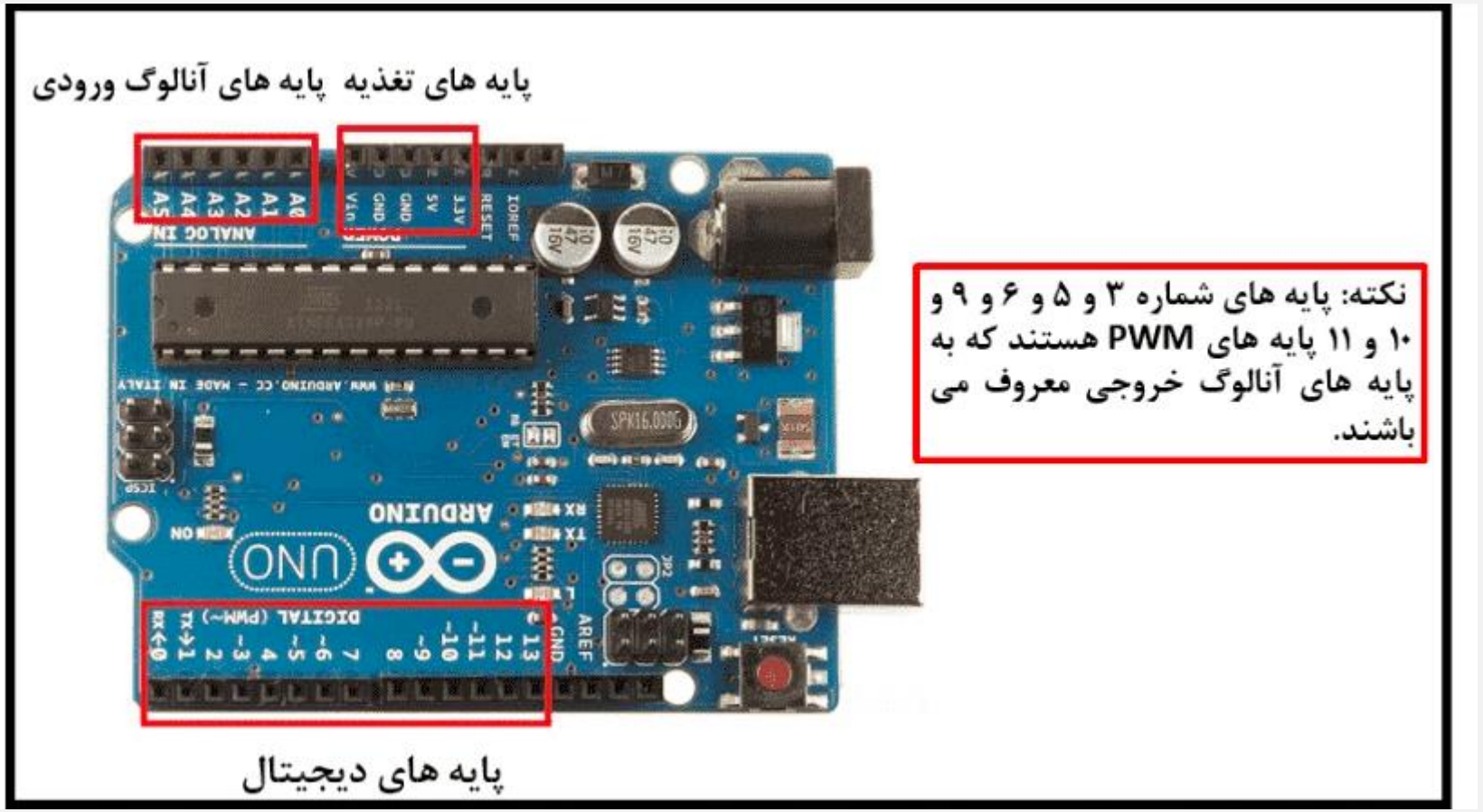
معرفی برخی از بخش های مختلف نرم افزار Arduino IDE

Tools

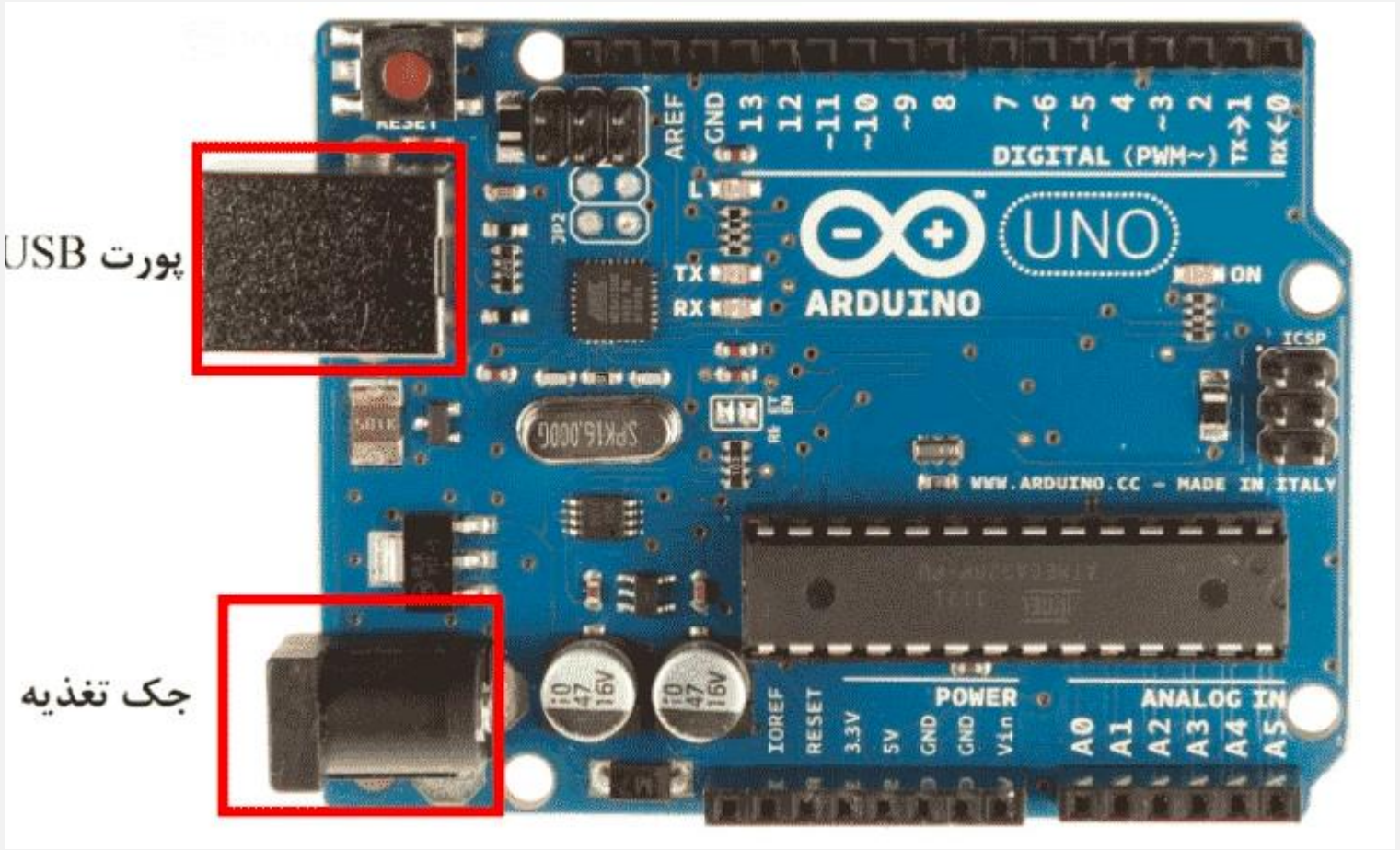
Auto Format: این گزینه ظاهر کد شما را مرتب می کند. به این معنی که با ایجاد تورفتگی ها و مرتب کردن آکلادها و پرانتزها و ...، ساختار کد شما مطابق استانداردهای کدونویسی شده و خوانایی کد بالا می رود.
Archive Sketch: یک کپی از اسکچ کنونی را در قالب . Zipدر همان مسیری که اسکچ در آن وجود دارد ذخیره می کند.
Serial Monitor: پنجره ی کنترل ارتباط سریال را باز می کند. با باز شده این پنجره، معمولا پردازنده ی شما یک بار ریست می شود.
Board: با استفاده از این بخش می توانید نوع بردی که در حال حاضر از آن استفاده می کنید را انتخاب کنید. همچنین می توانید بردهای جدید را به نرم افزار اضافه کنید.
Port: در این بخش می توانید پورتی که برد شما به آن متصل است را انتخاب کنید.
Programmer: اگر می خواهید پردازنده ی خود را با استفاده از یک پروگرامر خارجی برنامه ریزی کنید و نمی خواهید از مبدل USBبه سریال استفاده کنید، از این بخش می توانید پروگرامر خود را انتخاب کنید.
Burn Bootloader: بوت لودر را روی پردازنده می ریزد و فیوز های پردازنده را تنظیم می کند.

Help

در این منو می توانید به توضیحات و راهنمایی های بیشتر راجع به نرم افزار Arduino IDEدسترسی پیدا کنید.



این برد را می‌توانید با استفاده از USBکه به کامپیوتر وصل می‌شود و همچنین از طریق جک تغذیه که به آداپتور وصل می‌شود، تغذیه کنید.
پورت USBعلاوه بر این که برد را تغذیه می‌کند می‌تواند برد را از طریق کامپیوتر پروگرام کند.


پورت USB
جک تغذیه
DIGITAL (PWM~)
ARDUINO
UNO
ON
ICSP
WWW.ARDUINO.CC – MADE IN ITALY
POWER
ANALOG IN

تعریف متغیر در آردوینو

```
data_type variable_name = initial_value;

int myNumber = 42;          // Integer variable with an initial value of 42
float temperature = 25.5;   // Floating-point variable with an initial value of 25.5
char myChar = 'A';          // Character variable with an initial value of 'A'
boolean isOn = true;        // Boolean variable with an initial value of true

int anotherNumber;          // Integer variable without an initial value
```

for حلقه

```
void setup() {
  // Your setup code here
}

void loop() {
  // Your main code here

  // Example of a for loop counting from 0 to 4
  for (int i = 0; i < 5; i++) {
    // Code inside the loop, repeated 5 times
    // Use the variable 'i' to know the current iteration
    // For example, you can do something like: digitalWrite(ledPin, HIGH);
    delay(1000); // Wait for 1 second
  }

  // Your remaining code here
}
```

while حلقه

```
void setup() {
  // Your setup code here
}

void loop() {
  // Your main code here

  // Example of a while loop
  int counter = 0;
  while (counter < 5) {
    // Code inside the loop, repeated as long as counter is less than 5
    // Use the variable 'counter' to know the current iteration
    // For example, you can do something like: digitalWrite(ledPin, HIGH);
    delay(1000); // Wait for 1 second
    counter++;
  }

  // Your remaining code here
}
```

رفرنس برای کار با آردوینو

1- Programming Arduino: Getting Started with Sketches

2- Arduino Workshop: A Hands-On Introduction with 65 Projects 1st Edition

3- Arduino Programming in 24 Hours, Sams Teach Yourself

4- Arduino: Advanced Methods and Strategies of Using Arduino

5- Getting Started with Arduino: by Massimo Banzi and Michael Shiloh

گرامر زبان آردوینو

Notice the functions
Functions are capable of returning a value after they finish running but here setup and loop will return nothing since they have void in front of function name.
The functions happen between the curly braces.

```
void setup () {
    // put your setup code here, to run once:
    pinMode(13, OUTPUT);
}

void loop() {
    //put your main code here, to run repeatedly:
    digitalWright(13, HIGH);
    delay(1000);
    digitalWrite(13, LOW);
    delay(1000);
}
```

گرامر زبان آردوینو

Notice the lines are intended. You can not use indentation but it is a good style and well sorted.
In Arduino indent is two spaces.

```
void setup () {
  // put your setup code here, to run once:
  pinMode(13, OUTPUT);
}

void loop() {
  setup();  // not a good practice in Arduino because it is automatically called by arduino
  //put your main code here, to run repeatedly:
  digitalWright(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

گرامر زبان آردوینو

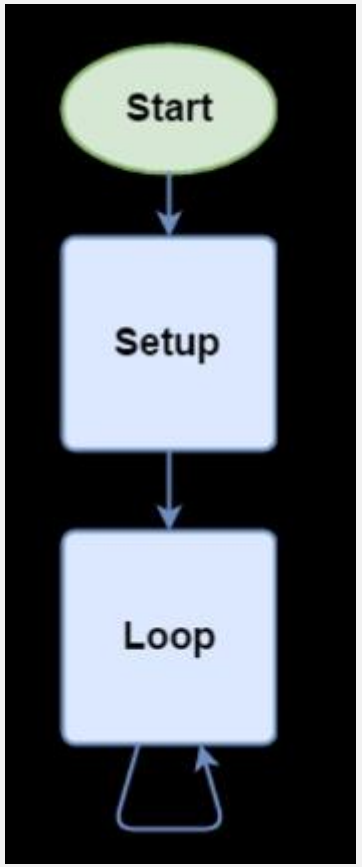
To check the syntax of your code you can click quick check.
C programs are executed sequentially which means our processor in our Arduino is executing one statement at a time before moving on to the next.

```
void setup () {
	// put your setup code here, to run once:
	pinMode(13, OUTPUT);
}

void loop() {
	setup();  // not a good practice in Arduino because it is automatically called by arduino
	//put your main code here, to run repeatedly:
	digitalWright(13, HIGH);
	delay(1000);
	digitalWrite(13, LOW);
	delay(1000);
}
```

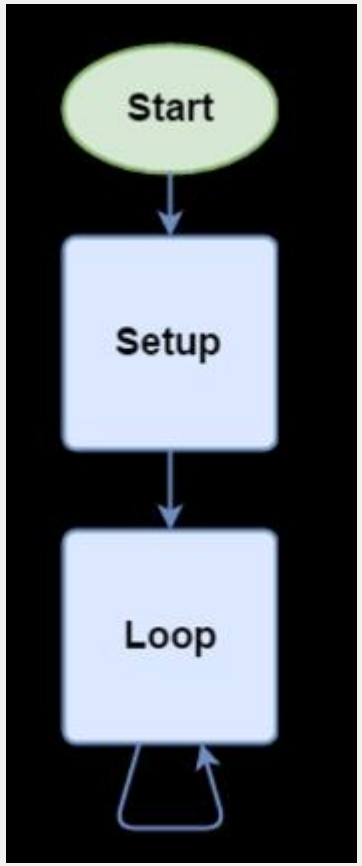
گرامر زبان آردوینو

All Arduino programs requires a setup and loop function



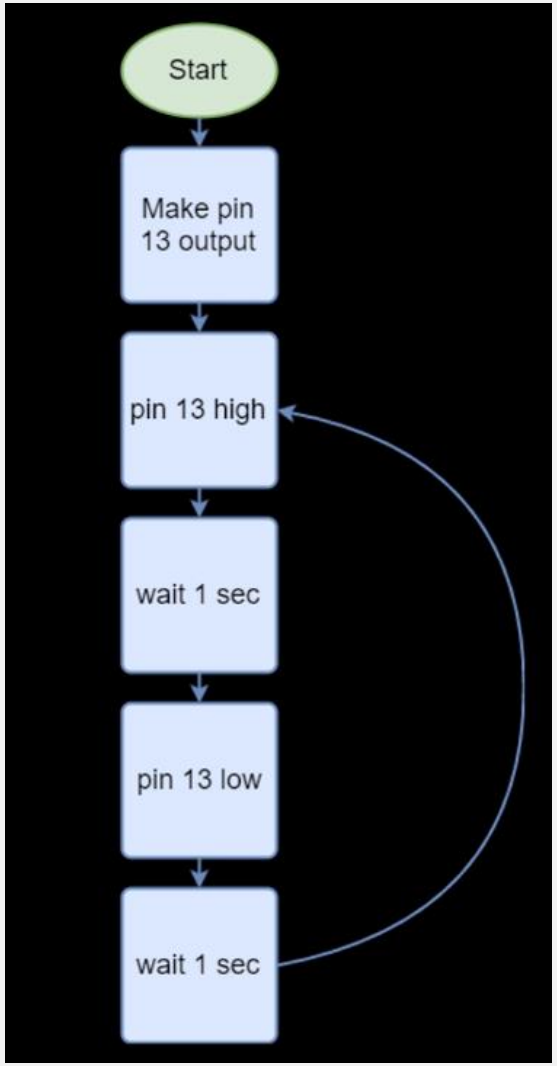
گرامر زبان آردوینو

All Arduino programs requires a setup and loop function
In our programs the setup functions acts as an entry point. This is where the processor enters our code and execution begins. After setup is run, everything in the loop function is executed and the loop function is run over and over again.



گرامر زبان آردوینو

If instead of showing the setup and loop function names, we explicitly wrote out each step, we would end up with a slightly longer flowchart. In this we see that the program starts with the code in setup setting pin 13 as an output in this case. We then move to the top of the loop function where the first statement is to set pin 13 to high, make the pin output be five volts. Then we delay for one second, make pin 13 low which is zero volts and then delay for another one second. After that, execution returns to the top of the loop function with setting pin 13 to high again..



Literals, variables and data types

Literal: is simply a fixed value that does not change throughout the program
Example: pinMode(13, OUTPUT)
In here value 13 is a literal which is an inteager.
It is considered a literal because the number 13 won't change while running our program.

Other literals include the floating point type, which can be in decimal form or scientific notation where the number after the e is the exponent part.
Example: float
3/14159
6/02E-19 = 1/602*10^(-19)

char
Ex: 'G'
string
"Good Job!"
bool
True or false

Literals, variables and data types

Literals and variables can be one of a few types

- void->It means no value
- int->2-byte integer: -32,768 to 32,767

The basic integer takes up two bytes in Arduino memory and as such can be a number between -32700 and 32767

- Float->4-byte floating point value -3/4*10^38 to 3/4*10^38
- Character is shortened to its first letters and is often pronounced care or char. Characters are stored as a one byte number Which means only 256 possible characters are available for us in the char type. 1-byte character 256 possibilities

Literals, variables and data types

Characters are stored as numbers in the Ascii format which is The American Standard Code for Information Interchange, which was created in the 1960s.

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	null	32	space	64	@	96	`
1		33	!	65	A	97	a
2		34	"	66	B	98	b
3		35	#	67	C	99	c
4		36	$	68	D	100	d
5		37	%	69	E	101	e
6		38	&	70	F	102	f
7		39	'	71	G	103	g
8		40	(	72	H	104	h
9	tab	41	)	73	I	105	i
10	line feed	42	*	74	J	106	j
11		43	+	75	K	107	k
12		44	,	76	L	108	l
13	carriage return	45	-	77	M	109	m
14		46	.	78	N	110	n
15		47	/	79	O	111	o
16		48	0	80	P	112	p
17		49	1	81	Q	113	q
18		50	2	82	R	114	r
19		51	3	83	S	115	s
20		52	4	84	T	116	t
21		53	5	85	U	117	u
22		54	6	86	V	118	v
23		55	7	87	W	119	w
24		56	8	88	X	120	x
25		57	9	89	Y	121	y
26		58	:	90	Z	122	z
27		59	;	91	[	123	{
28		60	<	92	\	124	|
29		61	=	93	]	125	}
30		62	>	94	^	126	~
31		63	?	95	_	127	delete

Apply data types to variables

A variable is a storage location paired with a symbolic name often called a reference or a label.
Bellow I may not be able to store number 3 in char container however I can store it in inteager container.

```
// now we will create a global scale for our int container
int led = 13;

void setup () {
	// put your setup code here, to run once:
	pinMode(led, OUTPUT);
}

void loop() {
	setup();  // not a good practice in Arduino because it is automatically called by arduino
	//put your main code here, to run repeatedly:
	digitalWright(led, HIGH);
	delay(1000);
	digitalWrite(led, LOW);
	delay(1000);
}
```

Apply data types to variables

What this example do?

```
// now we will create a global scale for our int container
int led = 13;

void setup () {
	// put your setup code here, to run once:
	pinMode(led, OUTPUT);
}

void loop() {
	setup();  // not a good practice in Arduino because it is automatically called by arduino
	//put your main code here, to run repeatedly:
	digitalWright(led, HIGH);
	delay(1000);
	digitalWrite(led, LOW);
	delay(1000);
	led = 12;
}
```

Apply data types to variables

Another example

```
// now we will create a global scale for our int container
int led = 13;

void setup () {
	// put your setup code here, to run once:
	pinMode(led, OUTPUT);
}

void loop() {
	setup();  // not a good practice in Arduino because it is automatically called by arduino
	//put your main code here, to run repeatedly:
	digitalWright(led, HIGH);
	delay(1000);
	digitalWrite(led, LOW);
	delay(1000);
	led = 12;
}
```

Arithmetic operations

Another example

There are six basic operators for performing simple math.

1. = (assignment)

This is used to set a variable's value and is quite different from the equals sign you might find in regular mathematics.

2. + (addition)

It is used to add two numbers together.

3. – (subtraction)

It is for subtracting one value from another.

4. * (multiplication)
5. / (division)
6. % (modulo)

It is used to calculate the remainder after the division of one number by another.

Arithmetic operations

Example!

```
int a;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600); // Initializing our serial communication3




}

void loop() {
  // put your main code here, to run repeatedly:
  a = 4 * 3 / ( 2 + 4 ) + 1;
  Serial.println(a);

}
```

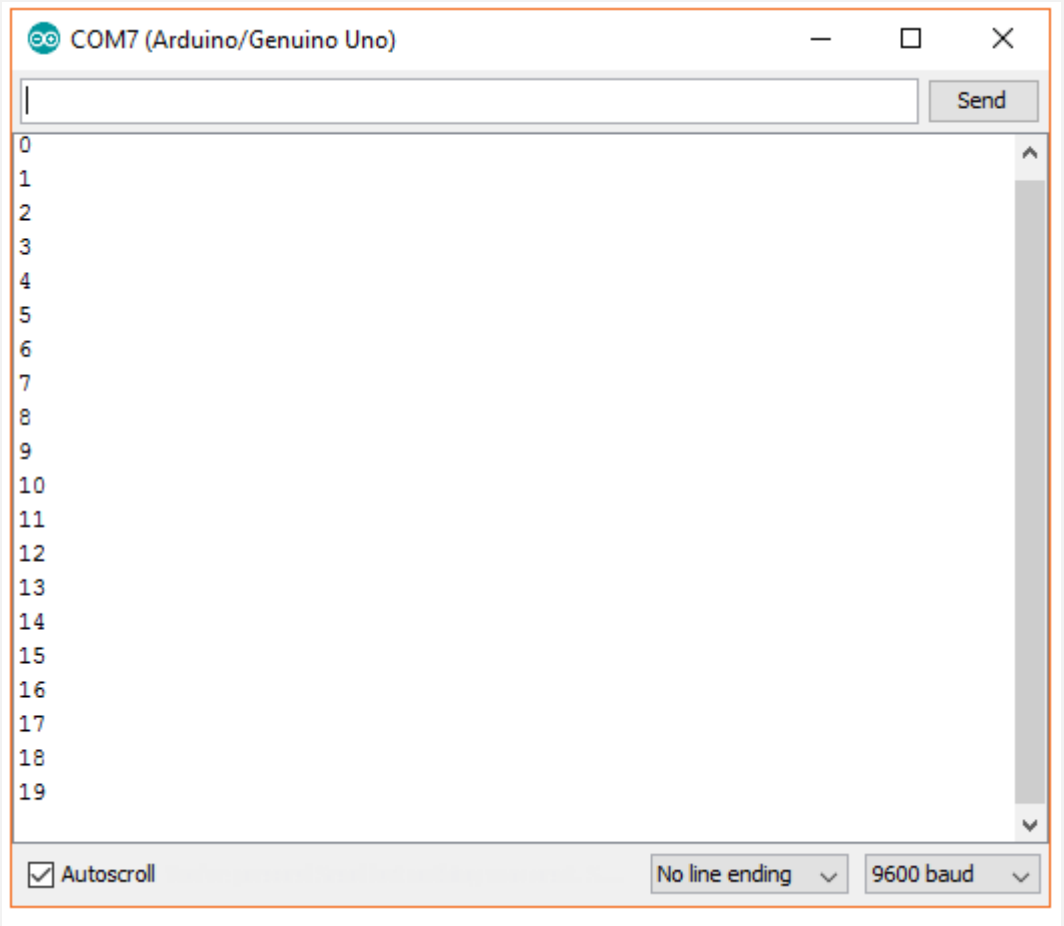
Shortcuts

Shortcut	Operation
a++	a = a + 1
a--	a = a - 1
a += 2	a = a + 2
a -= 3	a = a - 3
a *= 4	a = a * 4
a /= 2	a = a / 2
a %= 2	a = a % 2

practice

Write an Arduino program that starts with an integer at 0, prints it to the Serial Monitor, and increments the integer. It should continue printing and incrementing the integer forever.

Hint: you will want to add some kind of delay so that you can actually read the output. If you open the Serial Monitor, you should see the following:



مثال: راه اندازی یک پروژه ساده با آردوینو

ابتدا یک اسکچ جدید بسازید و آن را با نام دلخواه خود ذخیره کنید. سپس کد زیر که مربوط به چشمک زدن ال ای دی موجود روی برد آردوینو است را در محیط نرم افزار خود، کپی کنید.

```
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

توجه: اگر مبدل USB به سریال آردوینو شما، آی سی CH340Gاست، قبل شروع کار، درایور Ch340Gرا نصب کنید.

مثال: راه اندازی یک پروژه ساده با آردوینو

بعد از نوشتن کد، از منوی Toolsمدل برد و پورت آن را انتخاب کنید و سپس با زدن کلید آپلود، کد نوشته شده را روی برد خود آپلود کنید. اگر کد بدون خطا کامپایل شود و با موفقیت روی برد آپلود شود، باید پیام Done uploadingنمایش داده شود.